Erkennung ungenutzter Bibliotheken in Deployments zur risikobasierten Priorisierung von Sicherheitsbefunden

Projektarbeit 1 (T4_1000)

Im Rahmen der Prüfung:

Bachelor of Science (B. Sc.)

des Studienganges Informatik

an der
Dualen Hochschule Baden-Württemberg Karlsruhe

von

Abgabedatum	05.01.2026
Bearbeitungszeitraum	20.10.2025 - 05.01.2026
Matrikelnummer, Kurs	, TIN
Ausbildungsfirma	SAP SE Dietmar-Hopp-Allee 16 69190 Walldorf, Deutschland
Betreuer der Ausbildungsfirma	
Gutachter der Dualen Hochschule	—

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich meine Projektarbeit 1 (T4_1000) mit dem Thema:

> *Erkennung ungenutzter Bibliotheken in Deployments zur risikobasierten Priorisierung von Sicherheitsbefunden*

gemäß § 5 der „Studien- und Prüfungsordnung DHBW Technik“ vom 29. September 2017 selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

K, 26. Februar 2026

gez.: ,

Sperrvermerk

Die nachfolgende Arbeit enthält vertrauliche Daten der:

SAP SE
Dietmar-Hopp-Allee 16
69190 Walldorf
Deutschland

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anderslautende Genehmigung vom Dualen Partner vorliegt.

Erklärung zur Nutzung Künstlicher Intelligenz

Für die folgende Arbeit wurde die Unterstützung Künstlicher Intelligenz genutzt.
Gemini 2.5 Flash und Gemini 2.5 Pro wurden genutzt, um bei der Literatur Recherche und Fachterminologie Suche zu unterstützen und Rechtschreibung, Grammatik und Ausdruck zu verbessern.
Obwohl diese Hilfsmittel meine Fähigkeiten erweitert und zu meinen Erkenntnissen beigetragen haben, ist es wichtig darauf hinzuweisen das sie inhärente Einschränkungen aufweisen. Ich habe mich bemüht jegliche Unterstützung Künstlicher Intelligenz transparent und verantwortungsvoll zu nutzen.

Diese Erklärung wurde von mir übersetzt, jedoch automatisch von AI Usage Cards generiert. https://ai-cards.org

Abstract

- English -

Abstract1

Abstract

- Deutsch -

Abstract2

Inhaltsverzeichnis

Abbildungsverzeichnis ..... **IX**

Tabellenverzeichnis ..... **X**

1 Einleitung **1**
1.1 Motivation 1
1.2 Problemstellung und Forschungsfragen 2
1.3 Zielsetzung und Abgrenzung 2
1.4 Aufbau der Arbeit 2

2 Grundlagen **3**
2.1 Dependency Management 3
2.2 Software Bill of Materials (SBOM) 4
2.3 Bloated Dependencies 4
2.3.1 Definition und Abgrenzung 4
2.3.2 Kategorisierung 5
2.3.3 Entstehungsmechanismen 5
2.3.4 Empirische Evidenz 6
2.3.5 Konsequenzen 7
2.4 Vulnerability-Datenbanken und Dependency Scanner 7
2.4.1 CVE und die National Vulnerability Database 7
2.4.2 Von der Datenbank zum Scanner-Befund 8

3 Verwandte Arbeiten **10**
3.1 Dynamische Ansätze (Runtime-Tracing, Coverage) 10
3.2 Statische Ansätze (Call-Graph, Reachability) 10
3.3 Einordnung der Ansätze nach L0–L2 und Eignung für TypeScript/Node.js 10

4 Konzept und Designentscheidung **11**
4.1 Usage-Evidence-Schema 11
4.2 Anforderungen an den PoC 17
4.3 Bewertung und Methodenauswahl 17
4.4 Architektur der Gesamtpipeline 17

5 Implementierung **18**
5.1 Tooling-Stack und Projektstruktur 18
5.2 L0: Präsenzermittlung via Dependency-Tree/SBOM 18
5.3 L1: Loaded-Heuristiken 18
5.4 L2: Reachability-Analyse (Call-Graph, Entry Points, Mapping) 18
5.5 Ergebnisformat und Risk-Scoring 18
5.6 Beispiel-Durchlauf 18

6 Evaluation **19**
6.1 Setup und Testkorpus 19
6.2 Metriken und Vorgehen 19
6.3 Ergebnisse und Vergleich mit npm audit 19
6.4 Limitierungen (JS-Dynamik, Reflection, Bundling) 19

7 Fazit und Ausblick **20**
7.1 Fazit 20
7.2 Ausblick 20
Literaturverzeichnis ..... **a**

Abkürzungsverzeichnis

CPE **– Common Platform Enumeration** 8

CVE **– Common Vulnerabilities and Exposures** 7, 8, 9

CVSS **– Common Vulnerability Scoring System** 8, 9

CWE **– Common Weakness Enumeration** 8

NIST **– National Institute of Standards and Technology** 7

NVD **– National Vulnerability Database** 7, 8

OWASP **– Open Worldwide Application Security Project** 8

PoC **– Proof of Concept** 9

SBOM **– Software Bill of Materials** 8

SCA **– Software Composition Analysis** 8

TS **– TypeScript** 5

npm **– Node Package Manager** 8

Abbildungsverzeichnis

Abbildung 1 Dreistufiger Prüfprozess eines metadatenbasierten Dependency-Scanners (Inventarisierung, Matching, Meldung). 9
Abbildung 2 Trichtermodell: Jede Evidenzstufe beschreibt eine Teilmenge der vorherigen. Der Großteil installierter Dependencies erzeugt keine L1- oder L2-Evidenz. 13

Tabellenverzeichnis

Tabelle 1 Bloated Dependencies im Maven-Ökosystem [1]. 6
Tabelle 2 Bloated Dependencies im CommonJS/npm-Ökosystem [2]. 6
Tabelle 3 Evidenzstufen des Usage-Evidence-Schemas mit Analysemethode, FN-Klassen und empirischer Einordnung. 14

1 Einleitung

1.1 Motivation

Dependency Scanner wie npm audit und OWASP Dependency-Check sind aus der modernen Softwareentwicklung nicht mehr wegzudenken [3]. Diese Tools basieren primär auf Software Bill of Materials (SBOM) und operieren ausschließlich auf Package-Ebene [4]. Aktuelle Forschung zeigt jedoch fundamentale Limitierungen dieser Ansätze, die zu erheblichen praktischen Problemen führen.

Die Unfähigkeit zu erkennen, ob Code tatsächlich zur Laufzeit geladen wird, führt zu zwei kritischen Problemen. Erstens akkumulieren Projekte obsolete Dependencies, die erhebliche Wartungskosten verursachen [5]. Empirische Analysen zeigen, dass 75,1% aller Maven-Dependencies als „bloated“ klassifiziert werden können, wobei 57% der transitiven Dependencies vollständig ungenutzt sind [6]. Im JavaScript-Ökosystem ist die Situation vergleichbar: 50,6% der Dependencies in CommonJS-Packages sind bloated [2].

Diese obsoleten Dependencies entstehen durch verschiedene Mechanismen. Transitive Abhängigkeiten werden automatisch in die Dependency-Hierarchie eingefügt, ohne dass Entwickler sich deren Präsenz bewusst sind [5]. Darüber hinaus werden Dependencies während der Entwicklung hinzugefügt und bei Code-Refactorings nicht entfernt [2]. Das Entfernen ist jedoch oft schwierig, weil Entwickler nicht mit ausreichender Sicherheit beurteilen können, ob eine Dependency wirklich entbehrlich ist [5]. Die resultierenden Kosten manifestieren sich in erhöhten Binary-Größen, verlängerten Build-Zeiten, erhöhtem Speicherverbrauch und gesteigertem Sicherheitsrisiko durch eine vergrößerte Angriffsfläche [6], [7]. In containerisierten Deployments wird die Problematik besonders sichtbar, da Images teils eine große Menge veralteter oder verwundbarer JavaScript-Pakete enthalten [8].

Zweitens führt die Fokussierung auf Package-Level-Analysen zu einer hohen Rate an False-Positive-Warnungen. Die zentrale Herausforderung liegt in der fehlenden Granularität: SBOM-basierte Tools können nicht zwischen deployed und nicht-deployed Code differenzieren [9], noch können sie feststellen, ob vulnerable Funktionen tatsächlich vom Anwendungscode aufgerufen werden.

1.2 Problemstellung und Forschungsfragen

1.3 Zielsetzung und Abgrenzung

1.4 Aufbau der Arbeit

2 Grundlagen

2.1 Dependency Management

Modern entwickelte JavaScript- und TypeScript-Anwendungen stützen sich in hohem Maße auf externe Bibliotheken. Um diese zu verwalten, verwenden Entwickler den Package Manager npm (Node Package Manager). Ein Package Manager übernimmt drei zentrale Aufgaben: Er liest Manifest-Dateien wie `package.json`, lädt die deklarierten Bibliotheken aus dem zentralen npm-Registry herunter und installiert sie lokal im Projekt [10]. Dabei löst er nicht nur die explizit angeforderten Dependencies auf, sondern rekursiv auch alle deren Abhängigkeiten.

Dependencies lassen sich in zwei Kategorien einteilen. Direkte Dependencies werden explizit im Projekt-Manifest deklariert, beispielsweise `express` in der `package.json`. Der Anwendungscode importiert und nutzt diese Bibliotheken direkt. Transitive Dependencies hingegen sind Abhängigkeiten dieser direkten Dependencies [1]. Wenn ein Node.js-Projekt `express@4.18.2` deklariert, installiert npm automatisch auch die 31 Dependencies von express, darunter `body-parser`, `cookie` und `debug`. Alle installierten Packages werden im `node_modules`-Verzeichnis abgelegt.

Dieser Mechanismus führt zum Phänomen der Dependency Amplification: Eine einzelne direkte Dependency zieht eine Kaskade weiterer Dependencies nach sich [11]. Ein Projekt mit 50 direkten Dependencies kann so auf mehrere hundert installierte Packages anwachsen [11].

Bei TypeScript-Projekten wird diese Amplification zusätzlich verstärkt: Für jede JavaScript-Bibliothek ohne eingebaute Typdefinitionen muss ein separates Type-Definition-Package aus dem `@types/*`-Namespace installiert werden. Nutzt ein Projekt beispielsweise `express`, muss zusätzlich `@types/express` installiert werden, um TypeScript-Typen zu erhalten. npm unterscheidet zwischen `dependencies` (zur Laufzeit benötigt) und `devDependencies` (nur für Build/Test). Type-Definition-Packages werden als `devDependencies` deklariert, da sie ausschließlich zur Compile-Zeit erforderlich sind.

Diese Dependency Amplification führt zu einem fundamentalen Problem: Bei hunderten automatisch installierten Packages verlieren Entwickler den Überblick, welche Dependencies tatsächlich vom Anwendungscode genutzt werden. Die in Abschnitt 1.1 beschriebenen Konsequenzen (akkumulierte bloated dependencies, erhöhte Sicherheits-

risiken und False-Positive-Warnungen in Vulnerability Scannern) sind direkte Folgen dieses Mechanismus.

2.2 Software Bill of Materials (SBOM)

Wie in der Motivation beschrieben, basieren Scanner wie npm audit oder OWASP Dependency-Check auf Software Bills of Materials (SBOMs). Ein SBOM ist eine maschinenlesbare Liste aller Softwarekomponenten eines Produkts [12]. Im Gegensatz zu Manifest-Dateien wie `package.json`, die nur direkte Dependencies auflisten, erfasst ein SBOM auch alle transitiven Dependencies mit exakten Versionen. Die NTIA spezifiziert Mindestinhalte [13], das verbreitetste Format ist SPDX (ISO/IEC 5962:2021) [14].

SBOMs werden typischerweise durch spezialisierte Tools wie CycloneDX, Syft oder das Microsoft SBOM Tool aus den Dependency-Informationen eines Projekts generiert. Jeder SBOM-Eintrag dokumentiert eine Softwarekomponente mit folgenden Informationen: Paketname (z.B. `express`), Version (`4.18.2`), eindeutiger Identifikator als Package-URL (`pkg:npm/express@4.18.2`), Lizenzinformation (`MIT`), Lieferant sowie die Liste der direkten Dependencies dieser Komponente. Für ein typisches Node.js-Projekt mit 50 direkten Dependencies kann ein SBOM mehrere hundert Einträge umfassen, da auch alle transitiven Dependencies erfasst werden müssen.

Die fundamentale Einschränkung von SBOMs liegt im Detailgrad der Dokumentation: Dependencies werden auf Package-Ebene dokumentiert. Eine Bibliothek wird als Ganzes erfasst, unabhängig davon, welche ihrer Funktionen tatsächlich genutzt werden. Bindet ein Projekt die Bibliothek `lodash` ein und nutzt nur die Funktion `_.debounce()`, erfasst das SBOM lediglich `lodash@4.17.21` als Dependency, ohne zu dokumentieren, welche der über 300 verfügbaren Funktionen von lodash im Code aufgerufen werden.

2.3 Bloated Dependencies

2.3.1 Definition und Abgrenzung

Als **bloated** werden Dependencies bezeichnet, die im Deployment einer Anwendung installiert sind, aber zur Laufzeit nie verwendet werden [1]. Bestehende Analyse-Tools fokussieren primär auf veraltete (*outdated*) und verwundbare (*vulnerable*) Dependencies. Eine Dependency kann jedoch veraltet sein, ohne ungenutzt zu sein, und eine

verwundbare Dependency muss nicht zwingend bloated sein. Diese Arbeit ergänzt die bestehenden Dimensionen um den Aspekt der genutzten Codebasis.

2.3.2 Kategorisierung

Bloated Dependencies lassen sich nach ihrer Herkunft in zwei für npm relevante Typen einteilen. **Direkte** bloated Dependencies sind explizit im Manifest deklariert, werden jedoch nie vom Anwendungscode aufgerufen. **Transitive** bloated Dependencies wurden automatisch als Abhängigkeiten anderer Packages installiert, ohne dass die Anwendung sie je verwendet [1]. Transitive bloated Dependencies sind in der Praxis dominierend, da der Dependency-Auflösungsmechanismus sie automatisch und ohne direktes Zutun der Entwickler in den Abhängigkeitsbaum einfügt (vgl. Abschnitt 2.1). Im Maven-Ökosystem existiert zusätzlich die Kategorie **geerbter** Dependencies aus Parent-POMs, die in *TypeScript (TS)* keine Entsprechung hat und daher in dieser Arbeit nicht weiter betrachtet wird.

2.3.3 Entstehungsmechanismen

Bloated Dependencies entstehen durch verschiedene Mechanismen. Bei **direkten** Dependencies tritt häufig evolutionäre Akkumulation auf: Wenn Features umgebaut oder entfernt werden, bleiben die zugehörigen Packages oft im Manifest erhalten [5]. Bei **transitiven** Dependencies ist der Mechanismus fundamentaler: Sie werden automatisch installiert, sobald eine direkte Dependency sie benötigt. Entfernt man später Code, der diese direkte Dependency nutzte, muss die Deklaration manuell aus dem Manifest entfernt werden. Geschieht dies nicht, bleiben alle transitiven Dependencies dieser Dependency installiert, selbst wenn keine davon mehr genutzt wird (vgl. Abschnitt 2.1).

Das nachträgliche Entfernen ist für Entwickler riskant. Statische Analysetools melden zahlreiche False Positives, weil sie dynamische Sprachfeatures wie `eval()`, `require()` mit variablen Parametern oder dynamische Imports nicht vollständig erfassen können. Empirisch zeigt sich, dass Ansätze, die Call-Graph-Analyse mit einer Klassifikation der Dependency-Beziehungen kombinieren, rund ein Drittel dieser False Positives eliminieren können [5]. Eine explorative Studie mit 31 Pull Requests (23 davon beantwortet) deutet an, dass erhebliche Unsicherheiten besonders bei transitiven Dependencies bestehen: Während Entwickler 14 von 15 Vorschlägen zur Entfernung direkter bloated Dependencies akzeptierten, nahmen sie nur 4 von 8 Vorschläge zu transitiven Dependencies an [1].

2.3.4 Empirische Evidenz

Das Problem ist weit verbreitet. Im JavaScript-Ökosystem sind 50,6% aller Dependencies bloated [2]. Im Maven-Ökosystem fällt das Bild noch deutlicher aus: 75,1% aller Dependencies sind bloated, wie eine Analyse von 9.639 Artefakten zeigt [1]. Die Messgrößen unterscheiden sich dabei methodisch: Maven erfasst Dependency-Beziehungen als Kanten im Dependency-Graph (N = 723.444), npm hingegen einzelne Package-Installationen (N = 50.488). Die Prozentwerte sind daher nur bedingt direkt vergleichbar, zeigen aber in beiden Ökosystemen einen hohen Anteil ungenutzter Dependencies. Tabelle 1 und Tabelle 2 zeigen die Verteilung nach Typ.

Typ	**Anteil (N = 723.444)**
Direkt (bloated)	2,7%
Inherited (bloated)	15,4%
Transitiv (bloated)	57,0%
Gesamt bloated	**75,1%**

Tabelle 1: Bloated Dependencies im Maven-Ökosystem [1].

Typ	**Anzahl**	**Anteil der Kategorie**
Direkt (bloated)	120 / 869	13,8%
Indirekt (bloated)	25.446 / 49.619	51,3%
Gesamt bloated	**25.566 / 50.488**	**50,6%**

Tabelle 2: Bloated Dependencies im CommonJS/npm-Ökosystem [2].

Über beide Ökosysteme hinweg zeigt sich ein konsistentes Muster: Transitive Dependencies machen den Großteil des Bloats aus, während direkte Dependencies seltener betroffen sind. Im Maven-Ökosystem existiert zusätzlich die Kategorie geerbter Dependencies aus Parent-POMs (15,4% bloated), die in npm keine Entsprechung hat. Auf Projektebene ist die Verbreitung besonders hoch: 86,2% aller Maven-Artefakte und 98,9% aller untersuchten CommonJS-Packages enthalten mindestens eine bloated-transitive Dependency [1], [2].

Die Hebelwirkung des Entfernens lässt sich am Package `podcast-search` verdeutlichen [2]. Das Projekt hat zwei direkte Dependencies, von denen `npm` zur Laufzeit nie verwendet wird. Alle 679 transitiven Dependencies stammen ausschließlich von dieser einen ungenutzten Dependency. Das Entfernen einer einzigen Zeile in der `package.json` eliminiert so 680 von insgesamt 681 installierten Dependencies, ohne Funktionsverlust.

2.3.5 Konsequenzen

Dependency Bloat hat technische, wirtschaftliche und sicherheitskritische Folgen. Überflüssige Packages erhöhen die Binary-Größe, verlängern Installations- und Build-Zeiten und steigern den Speicherbedarf. In containerisierten Deployments, wo TypeScript-Anwendungen typischerweise betrieben werden, wird dies besonders sichtbar: Analysen offizieller Docker-Images zeigen, dass alle untersuchten Images verwundbare npm-Packages enthalten, im Durchschnitt 16,6 Security-Vulnerabilities pro Container [8].

Dependency-Scanner erkennen diese Vulnerabilities unabhängig von der tatsächlichen Erreichbarkeit des betroffenen Codes. Sicherheitskritisch ist dabei vor allem die vergrößerte Angriffsfläche: Jede zusätzlich installierte Bibliothek kann bekannte Schwachstellen (CVEs) einbringen. SBOM-basierte Scanner erfassen eine Bibliothek als Ganzes, ohne zu berücksichtigen, welche ihrer Funktionen tatsächlich aufgerufen werden. Ponta et al. zeigen, dass erst eine code-zentrische, nutzungsbasierte Analyse feststellen kann, ob vulnerable Funktionen im konkreten Anwendungskontext tatsächlich erreichbar sind [4]. Eine bloated Dependency mit einer bekannten Schwachstelle löst daher eine Warnung aus, obwohl kein Codepfad die Schwachstelle je erreichen könnte. Die Reduktion von Dependency Bloat ist damit nicht nur eine Maintenance-Maßnahme, sondern eine Voraussetzung für präziseres Vulnerability-Management.

2.4 Vulnerability-Datenbanken und Dependency Scanner

Dieses Kapitel beschreibt den Aufbau der Schwachstellendatenbanken, auf denen Dependency-Scanner basieren, und den konkreten Prüfprozess, der zu einem Scanner-Befund führt. Beides bildet die technische Grundlage, um nachzuvollziehen, warum metadatenbasierte Scanner nicht zwischen erreichbaren und nicht erreichbaren Schwachstellen unterscheiden können.

2.4.1 CVE und die National Vulnerability Database

Das *Common Vulnerabilities and Exposures (CVE)*-Programm ist der zentrale Standard zur einheitlichen Benennung öffentlich bekannter Sicherheitslücken [15]. Jede Schwachstelle erhält einen eindeutigen Bezeichner in der Form CVE-JAHR-NUMMER, der eine ökosystemübergreifende Referenzierung ermöglicht. Die *National Vulnerability Database (NVD)* des *National Institute of Standards and Technology (NIST)* baut auf diesem Standard auf und reichert jeden *CVE*-Eintrag mit strukturierten Metadaten

an: einer Schweregradbewertung nach *Common Vulnerability Scoring System (CVSS)*, einer Schwachstellentyp-Klassifikation nach *Common Weakness Enumeration (CWE)* sowie einer Liste betroffener Produkte nach *Common Platform Enumeration (CPE)* [16]. Anwar et al. erfassten in einem Snapshot von Mai 2018 über 107.200 *CVE*-Einträge, die der *NVD* über zwei Jahrzehnte hinzugefügt wurden [16].

CPE ist ein standardisiertes Namensschema zur eindeutigen Identifikation von Software-Produkten und -Versionen. Der *CPE*-Eintrag ist für *NVD*-basierte Dependency-Scanner das entscheidende Matching-Feld: Er codiert betroffene Produkte im Format `cpe:2.3:a:HERSTELLER:PRODUKT:VERSION` und definiert damit, welche Package-Versionen als verwundbar gelten [16]. Allerdings verwenden *CPE*-Bezeichner eine andere Granularität und Konvention als Package-Repository-Koordinaten, was zu ungenauem Package-Mapping führt [9]. Metadatenbasierte Scanner prüfen zudem nicht, ob die vulnerable Funktion einer Bibliothek im konkreten Anwendungskontext tatsächlich aufgerufen wird; das ist der zentrale Unterschied zu code-zentrischen Ansätzen [3].

2.4.2 Von der Datenbank zum Scanner-Befund

Software Composition Analysis (SCA)-Tools wie `npm audit`, das in *Node Package Manager (npm)* integrierte Kommandozeilenwerkzeug zur Sicherheitsprüfung von Abhängigkeiten, oder *Open Worldwide Application Security Project (OWASP)* Dependency-Check, ein verbreitetes Open-Source-Tool der Open Worldwide Application Security Project Foundation, erzeugen automatisch ein Inventar aller eingesetzten Open-Source-Komponenten und gleichen dieses gegen Schwachstellendatenbanken ab [17]. Dabei unterscheiden sich die genutzten Datenquellen je nach Tool: *OWASP* Dependency-Check bezieht Schwachstellendaten unter anderem aus der *NVD* und nutzt *CPE*-Matching zur Identifikation betroffener Packages [17], während `npm audit` seit Oktober 2021 auf die GitHub Advisory Database mit Package-URL (purl) als Identifikator setzt [18]. Trotz unterschiedlicher Datenquellen lässt sich der Ablauf auf drei grundlegende Schritte reduzieren [17]:

1. **Inventarisierung:** Der Scanner liest Dependency-Manifest-Dateien wie `package-lock.json` und erzeugt daraus eine Liste aller Packages mit exakten Versionen [17]. Alternativ kann ein *Software Bill of Materials (SBOM)* (vgl. Abschnitt 2.2) als Eingabequelle dienen, das den in Abschnitt 2.1 beschriebenen Dependency-Tree in maschinenlesbarer Form abbildet.

2. **Matching:** Jedes Package wird gegen die Einträge der jeweiligen Datenbank abgeglichen. Ein Treffer liegt vor, wenn Paketname und Version in den betroffenen Versionsbereich eines Schwachstellen-Eintrags fallen.
3. **Meldung:** Jeder Treffer wird als Befund ausgegeben, inklusive *CVE*-ID, *CVSS*-Score und betroffener Version.

Abbildung 1: Dreistufiger Prüfprozess eines metadatenbasierten Dependency-Scanners (Inventarisierung, Matching, Meldung).

Ausschließlich Paketname und Versionsnummer bestimmen das Ergebnis. Ob die vulnerable Funktion im konkreten Anwendungskontext erreichbar ist, geht in diesen Prozess nicht ein (vgl. die in Abschnitt 2.3 quantifizierten Limitierungen). Genau diese Lücke adressiert der *Proof of Concept (PoC)* dieser Arbeit.

3 Verwandte Arbeiten

3.1 Dynamische Ansätze (Runtime-Tracing, Coverage)

3.2 Statische Ansätze (Call-Graph, Reachability)

3.3 Einordnung der Ansätze nach L0–L2 und Eignung für TypeScript/Node.js

4 Konzept und Designentscheidung

Dieses Kapitel entwickelt das konzeptuelle und methodische Fundament des PoC in drei aufeinander aufbauenden Schritten. Zunächst wird ein gestuftes Evidenzmodell eingeführt, das die in Abschnitt 2.4 und Abschnitt 2.3 beschriebenen Limitierungen metadatenbasierter Scanner formalisiert; es bildet den Bewertungsrahmen für alle nachfolgenden Entscheidungen. Auf dieser Grundlage werden die funktionalen Anforderungen an den PoC abgeleitet und die methodische Entscheidung für eine statische Erreichbarkeitsanalyse (L2a) begründet. Abschließend wird die Gesamtarchitektur der Analysepipeline beschrieben, die diese Entscheidungen in eine konkrete Implementierungsstruktur überführt.

4.1 Usage-Evidence-Schema

Metadatenbasierte Scanner setzen die Präsenz einer Dependency im Deployment-Artefakt mit ihrer potenziellen Exploitierbarkeit gleich. Diese Annahme ist empirisch nicht haltbar: Installiert zu sein ist eine notwendige, aber keine hinreichende Bedingung dafür, dass eine verwundbare Funktion im konkreten Anwendungskontext tatsächlich erreichbar ist. Der in Abschnitt 2.4 beschriebene Scanner-Prozess prüft ausschließlich Paketname und Version gegen Schwachstellendatenbanken, ohne zu berücksichtigen, ob die betroffene Funktion vom Anwendungscode je aufgerufen wird. Gleichzeitig zeigt Abschnitt 2.3, dass im npm-Ökosystem 50,6% aller installierten Dependencies zur Laufzeit nie verwendet werden. In Kombination führt dies dazu, dass Warnungen für Dependencies gemeldet werden, die zwar installiert, aber nie referenziert oder erreichbar sind. Das in Abschnitt 1.1 beschriebene Grundproblem, die fehlende Erreichbarkeitsanalyse metadatenbasierter Scanner, lässt sich damit als fehlende Differenzierung zwischen drei Evidenzstufen formalisieren. Diese Stufen werden im Folgenden als L0 (Präsenz), L1 (Import) und L2 (Erreichbarkeit) eingeführt. L0 und L2 sind in der Forschungsliteratur verankert [4], [19]; die explizite Formalisierung von L1 als eigenständiger Evidenzstufe sowie die Differenzierung von L2 in L2a und L2b sind Beiträge dieser Arbeit.

Dass eine reine Präsenzanalyse unzureichend ist, belegen drei unabhängige Forschungsgruppen aus verschiedenen Perspektiven. Das Grundproblem formulieren Ponta et al. konzeptionell: Ob eine verwundbare Funktion im konkreten Anwendungskontext tatsächlich aufgerufen wird, lässt sich ausschließlich durch eine nutzungsbasierte Analyse feststellen, nicht durch den bloßen Abgleich von Paketname und Version [4]. Wie

gravierend die Folgen ohne eine solche Analyse sind, zeigt dieselbe Forschungsgruppe empirisch: 88,8% der von OWASP Dependency-Check gemeldeten Befunde sind False Positives, wenn tatsächliche Code-Erreichbarkeit als Maßstab gilt [3]. Imtiaz et al. bestätigen dies aus einer anderen Richtung: Selbst Steady, ein code-zentrischer Scanner, der bereits Erreichbarkeit berücksichtigt, meldet bei 84,2% seiner Befunde Packages, die im Anwendungskontext gar nicht genutzt werden, was zeigt, dass das Problem auch innerhalb fortgeschrittener Tools fortbesteht [17]. Pashchenko et al. ergänzen eine dritte Perspektive: Sie zeigen, dass bereits die Unterscheidung zwischen installierten und tatsächlich deployten Dependencies ausreicht, um für manche Projekte die Warnungen für Development-only-Dependencies gegenüber den deployten um das Dreifache zu reduzieren [9]. Alle drei Befunde zeigen, dass das Problem auf verschiedenen Ebenen auftritt: auf der Ebene einzelner Funktionen, auf der Ebene ganzer Packages und auf der Ebene der Deployment-Struktur. Sie motivieren damit ein Modell, das zwischen Präsenz, syntaktischer Referenzierung und tatsächlicher Erreichbarkeit unterscheidet.

Für das Schema gelten folgende Begriffe. Als **Konstrukt** gilt jede referenzierbare Einheit des Laufzeit-relevanten Interfaces einer Dependency, also exportierte Funktionen, Klassen, Objekte und Konstanten; TypeScript- `type`- und `interface`-Deklarationen bleiben ausgeklammert. Side-Effect- Imports der Form `import 'reflect-metadata'` begründen L1-Evidenz, sind aber nicht auf L2-Niveau einordenbar, da sie kein adressierbares Konstrukt exponieren; ihre Behandlung in der Priorisierungslogik wird am Ende dieses Abschnitts erläutert. Als **Anwendungscode** gilt der Teil des Projekts außerhalb des `node_modules`-Verzeichnisses (vgl. Abschnitt 2.1). Das Schema erhebt keinen Vollständigkeitsanspruch: Jede Stufe erzeugt strukturell unvermeidbare False Negatives. Konkret bedeutet das: Dynamische Imports der Form `require(variable)` sind auf L1-Niveau nicht erfassbar, weil der Modulpfad erst zur Laufzeit bekannt ist. Dependency-Injection- Mechanismen wie NestJS-Dekoratoren laden Module ohne klassische Importanweisung und entziehen sich damit der L2a-Analyse. Diese und weitere strukturelle Grenzen werden in den Verwandten Arbeiten (Kapitel 3) methodisch und in der Evaluation (Kapitel 6) als Limitation des PoC behandelt.

Abbildung 2 veranschaulicht, dass jede Stufe eine echte Teilmenge der vorherigen beschreibt: Nicht jede installierte Dependency wird importiert, und nicht jede importierte Dependency besitzt einen nachweisbaren Aufrufpfad.

Abbildung 2: Trichtermodell: Jede Evidenzstufe beschreibt eine Teilmenge der vorherigen. Der Großteil installierter Dependencies erzeugt keine L1- oder L2-Evidenz.

Tabelle 3 fasst alle vier Stufen zusammen. Die Spalte **FN-Klasse** beschreibt, welche Arten von Dependencies eine Stufe strukturell nicht erfassen kann, also Fälle, in denen eine tatsächlich genutzte Dependency fälschlicherweise als nicht genutzt eingestuft wird (False Negative). Die Spalte **Empirischer Mehrwert** gibt an, welchen messbaren Vorteil eine Stufe gegenüber der reinen Präsenzanalyse (L0) liefert, entweder durch belegte Zahlen aus der Literatur oder durch den Hinweis, dass der Wert im Rahmen dieser Arbeit erstmals erhoben wird.

Stufe	**Frage**	**Methode**	**FN-Klasse**	**Empirischer Mehrwert**
L0 – Präsenz	Ist die Dependency installiert?	SBOM, Dependency-Tree (vgl. Abbildung 1)	Keine FNs innerhalb des statischen Trees	Baseline; Gesamtpräzision von npm audit: 24% (N=12) [19]
L1 – Import	Wird die Dependency referenziert?	Statische Quellcodeanalyse der Imports	Dynamische Imports ohne literalen Pfad	Obere Schranke der Exposition; isolierter Mehrwert in Kap. 6 erhoben
L2a – statisch	Ist ein Konstrukt erreichbar?	Aufrufstrukturanalyse (vgl. Kapitel 3)	Reflection, DI, glob. Augmentierungen	81% FP-Reduktion ggü. npm audit (N=12) [19]
L2b – dynamisch	Wird ein Konstrukt ausgeführt?	Laufzeitbeobachtung (vgl. Kapitel 3)	Unvollständige Testabdeckung	Präziseste Evidenz; im PoC nicht implementiert

Tabelle 3: Evidenzstufen des Usage-Evidence-Schemas mit Analysemethode, FN-Klassen und empirischer Einordnung.

L0 – Präsenz: Eine Dependency ist im statisch installierten Dependency-Tree vorhanden (vgl. Abbildung 1). L0 erzeugt zwei FP-Klassen: transitiv installierte, aber nie referenzierte Dependencies, adressierbar durch L1, sowie referenzierte Dependencies ohne erreichbaren Aufrufpfad zu verwundbaren Konstrukten, adressierbar durch L2. FNs sind innerhalb des statisch installierten Trees strukturell ausgeschlossen. Das Problem lässt sich am Beispiel `writex` konkret zeigen: npm audit meldet fünf Schwachstellenwarnungen (Advisories), von denen nach manueller Prüfung nur eine tatsächlich relevant ist, da die verwundbaren Funktionen der übrigen vier vom Anwendungscode nie aufgerufen werden [19]. Die Gesamtpräzision von npm audit über den gesamten Testkorpus ist Tabelle 3 zu entnehmen.

L1 – Import: Eine Dependency gilt als L1-positiv, wenn der Quellcode sie durch einen statischen `import`- oder `require()`-Aufruf mit literalem Modulpfad referenziert. Eingeschlossen sind außerdem Re-Exports der Form `export { x } from 'dependency'` sowie Namespace-Imports der Form `import * as x from 'dependency'`. Bei Namespace-Imports ist das konkret genutzte Konstrukt aus dem Import-Statement allein nicht bestimmbar, was die anschließende L2-Analyse erschwert.

Das zentrale Problem von L1 lässt sich an einem konkreten Beispiel zeigen: Eine Datei `legacy-feature.ts` importiert `lodash`, wird aber selbst nie vom Einstiegspunkt der Anwendung aus aufgerufen, weil das Feature bereits entfernt wurde. `lodash` ist damit L1-positiv, obwohl zur Laufzeit kein Aufrufpfad existiert. L1 kann nicht unterscheiden, ob eine importierende Datei tatsächlich erreichbar ist oder nicht. Die Folge: L1 überschätzt die Menge tatsächlich genutzter Dependencies systematisch. Gleichzeitig ist diese Eigenschaft nützlich: Alle Dependencies, die L1-negativ sind, also nirgendwo im Quellcode importiert werden, sind unter der Annahme ausschließlich statischer Imports nicht über Anwendungslogik erreichbar. L1 bildet damit innerhalb dieses Geltungsbereichs eine konservative obere Schranke der tatsächlich genutzten Dependencies.

Als formale Evidenzstufe ist L1 in der Forschungsliteratur nicht beschrieben, obwohl Werkzeuge wie `depcheck` [20], das in der Analyse von Liu et al. als repräsentatives Import-Analyse-Werkzeug herangezogen wird [2], oder `knip` [21] das Prinzip implizit einsetzen: Sie melden installierte Dependencies, die im Quellcode nicht referenziert werden, beziehen diese Analyse jedoch nicht auf Vulnerability-Daten. Der isolierte Mehrwert von L1 gegenüber L0, also der Anteil der Dependencies, der zwar installiert aber nie importiert ist, wird in der Evaluation (Kapitel 6) erstmals gemessen.

L2 – Erreichbarkeit: L2a bezeichnet den durch Aufrufstrukturanalyse nachweisbaren Pfad vom Einstiegspunkt der Anwendung zu einem Konstrukt; ob dieser Pfad zur Laufzeit tatsächlich durchlaufen wird, bleibt dabei offen. Nielsen et al. messen für L2a eine FP-Reduktion von 81% gegenüber npm audit; diese Zahl basiert auf einem Testkorpus von N=12 Anwendungen und ist daher als Richtwert zu verstehen, nicht als allgemeingültiger Wert [19]. **L2b** bezeichnet durch Laufzeitbeobachtung bestätigte Ausführung und schließt damit die Lücke, die L2a offenlässt; L2b liefert die präziseste verfügbare Evidenz, ist im PoC jedoch nicht implementiert. Die methodischen Ansätze beider Unterebenen werden in den Verwandten Arbeiten (Kapitel 3) eingeordnet.

Die Implikationskette L2 → L1 → L0 gilt unter der Annahme statischer Modul-Referenzen. Vier Ausnahmeklassen können sie durchbrechen: dynamische Imports, Condi-

tional-Requires, DI-Mechanismen wie NestJS-Dekoratoren sowie `peerDependencies` über Plugin-Mechanismen. Wie verbreitet diese Ausnahmen in TypeScript/Node.js-Projekten sind, ist in der Literatur nicht systematisch gemessen. Für den PoC bedeutet das: Die Praxistauglichkeit von L2a hängt davon ab, wie dominant statische Imports im jeweiligen Testkorpus sind; diese Einschränkung wird in der Evaluation (Kapitel 6) als Limitation dokumentiert.

Für das Vulnerability-Management ergibt sich eine risikobasierte Priorisierungslogik für L0- bis L2a-Befunde. Fehlt L1-Evidenz, existiert unter der Annahme statischer Imports kein Referenzierungspfad; der Befund erhält niedrigste Priorität. Side-Effect-Imports wie `import 'reflect-metadata'` stellen einen Sonderfall dar: Sie begründen L1-Evidenz ohne adressierbares Konstrukt und erhalten daher ebenfalls mittlere Priorität, da ihre Sicherheitsrelevanz im Schema nicht bewertet werden kann. Liegt allgemeine L1-, aber keine L2a-Evidenz vor, ist die Dependency aktiv eingebunden, sodass ein Aufrufpfad über dynamische Mechanismen nicht ausgeschlossen werden kann; der Befund erhält mittlere Priorität. Mit L1- und L2a-Evidenz existiert ein nachweisbarer Aufrufpfad zu einem verwundbaren Konstrukt; der Befund erhält hohe Priorität. L2b-Evidenz würde eine höchste Klasse begründen, da sie dynamisch bestätigte Ausführung nachweist; diese Klasse ist im PoC nicht implementiert und wird daher in der Priorisierungslogik nicht weiter berücksichtigt. Die technische Umsetzung der Priorisierungslogik dokumentiert die Implementierung (Kapitel 5).

Der Beitrag dieser Arbeit liegt in drei Leistungen: erstens der Formalisierung von L1 als eigenständiger Evidenzstufe, wobei es sich um eine Kategorie handelt, die Werkzeuge wie `depcheck` [20] und `knip` [21] implizit umsetzen, ohne sie formal zu beschreiben oder gegen Vulnerability-Daten zu beziehen; zweitens der konzeptuellen Differenzierung von L2 in L2a und L2b, die bestehende Forschungsansätze präziser einordnet und den Implementierungsumfang des PoC klar abgrenzt; und drittens der prototypischen Implementierung von L0, L1 und L2a für das TypeScript/Node.js-Ökosystem. Kapitel 3 (Verwandte Arbeiten) ordnet bestehende Ansätze anhand dieses Schemas ein.

4.2 Anforderungen an den PoC

4.3 Bewertung und Methodenauswahl

4.4 Architektur der Gesamtpipeline

5 Implementierung

5.1 Tooling-Stack und Projektstruktur

5.2 L0: Präsenzermittlung via Dependency-Tree/SBOM

5.3 L1: Loaded-Heuristiken

5.4 L2: Reachability-Analyse (Call-Graph, Entry Points, Mapping)

5.5 Ergebnisformat und Risk-Scoring

5.6 Beispiel-Durchlauf

6 Evaluation

6.1 Setup und Testkorpus

6.2 Metriken und Vorgehen

6.3 Ergebnisse und Vergleich mit npm audit

6.4 Limitierungen (JS-Dynamik, Reflection, Bundling)

7 Fazit und Ausblick

7.1 Fazit

7.2 Ausblick

Literaturverzeichnis

[1] C. Soto-Valero, M. Monperrus, und B. Baudry, „A comprehensive study of bloated dependencies in the Maven ecosystem“, *Empirical Software Engineering*, Bd. 26, Nr. 4, S. 1–36, 2021, doi: 10.1007/s10664-020-09914-8.

[2] Y. Liu, D. Tiwari, C. Bogdan, und B. Baudry, „Detecting and removing bloated dependencies in CommonJS packages“, *Journal of Systems and Software*, Bd. 230, S. 112509, Dez. 2025, doi: 10.1016/j.jss.2025.112509.

[3] S. E. Ponta, H. Plate, und A. Sabetta, „Detection, assessment and mitigation of vulnerabilities in open source dependencies“, *Empirical Software Engineering*, Bd. 25, Nr. 5, S. 3175–3215, 2020, doi: 10.1007/s10664-020-09830-x.

[4] S. E. Ponta, H. Plate, und A. Sabetta, „Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software“, in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2018, S. 449–460. doi: 10.1109/ICSME.2018.00054.

[5] C.-C. Chuang, L. Cruz, R. van Dalen, V. Mikovski, und A. van Deursen, „ Removing dependencies from large software projects: are you really sure? “, in *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, Los Alamitos, CA, USA: IEEE Computer Society, Okt. 2022, S. 105–115. doi: 10.1109/SCAM55253.2022.00017.

[6] C. Soto-Valero und others, „A comprehensive study of bloated dependencies in the Maven ecosystem“, *Empirical Software Engineering*, 2021, doi: 10.1007/s10664-020-09914-8.

[7] C. Soto-Valero, T. Durieux, N. Harrand, und B. Baudry, „Coverage-based debloating for java bytecode“, *ACM Transactions on Software Engineering and Methodology*, Bd. 32, Nr. 2, S. 1–34, 2023.

[8] A. Zerouali, V. Cosentino, T. Mens, G. Robles, und J. M. Gonzalez-Barahona, „On the Impact of Outdated and Vulnerable Javascript Packages in Docker Images“, in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2019, S. 619–623. doi: 10.1109/SANER.2019.8667984.

[9] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, und F. Massacci, „Vuln4Real: A Methodology for Counting Actually Vulnerable Dependencies“, *IEEE Transac-*

tions on Software Engineering, Bd. 48, Nr. 5, S. 1592–1609, 2022, doi: 10.1109/TSE.2020.3025443.

[10] A. Decan, T. Mens, und M. Claes, „An empirical comparison of dependency network evolution in seven software packaging ecosystems“, *Empirical Software Engineering*, Bd. 24, Nr. 1, S. 381–416, 2019, doi: 10.1007/s10664-017-9589-y.

[11] R. Kikas, R. Söre, M. Dumas, und D. Pfahl, „Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem“, in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, ACM, 2022, S. 220–231. doi: 10.1145/3510003.3510142.

[12] National Telecommunications and Information Administration, „Software Bill of Materials (SBOM)“. 2021.

[13] National Telecommunications and Information Administration, „The Minimum Elements for a Software Bill of Materials (SBOM)“. [Online]. Verfügbar unter: https://www.ntia.gov/files/ntia/publications/sbom_minimum_elements_report.pdf

[14] International Organization for Standardization und International Electrotechnical Commission, „Information technology — SPDX®“. [Online]. Verfügbar unter: https://www.iso.org/standard/81870.html

[15] CVE Program, „Common Vulnerabilities and Exposures (CVE)“. 2024.

[16] A. Anwar, A. Abusnaina, S. Chen, F. Li, und D. Mohaisen, „Cleaning the NVD: Comprehensive Quality Assessment, Improvements, and Analyses“, *IEEE Transactions on Dependable and Secure Computing*, Bd. 19, Nr. 5, S. 3368–3382, 2022, doi: 10.1109/TDSC.2021.3125270.

[17] N. Imtiaz, S. Thorn, und L. Williams, „A Comparative Study of Vulnerability Reporting by Software Composition Analysis Tools“, in *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, in ESEM '21. New York, NY, USA: Association for Computing Machinery, 2021, S. 1–11. doi: 10.1145/3475716.3475769.

[18] GitHub, „GitHub Advisory Database Now Powers npm audit“. [Online]. Verfügbar unter: https://github.blog/security/supply-chain-security/github-advisory-database-now-powers-npm-audit/

[19] B. B. Nielsen, M. T. Torp, und A. Møller, „Modular call graph construction for security scanning of Node.js applications“, in *Proceedings of the 30th ACM*

SIGSOFT International Symposium on Software Testing and Analysis, in ISSTA 2021. Virtual, Denmark: Association for Computing Machinery, 2021, S. 29–41. doi: 10.1145/3460319.3464836.

[20] depcheck contributors, „depcheck: Check your npm module for unused dependencies“. [Online]. Verfügbar unter: https://github.com/depcheck/depcheck

[21] A. Boshuizen, „knip: Find unused files, exports and dependencies in your project“. [Online]. Verfügbar unter: https://knip.dev/